

Mini-projet GLPI — CY Tech

ING2 — Bases de données avancées — Année 2025–2026

Timothée François

Fiorina Roycee Raveendran

Hadjer Nedjar

17 mai 2026

Dépôt : <https://github.com/timfrncs/ing2-advanced-db-glpi-redesign>

Instance : Oracle 21c XE (PDB XEPDB1) — schéma GLPI_OWNER

Table des matières

1	Contexte et périmètre	3
2	Reverse engineering	3
2.1	Sources et périmètre analysé	3
2.2	Parcours des ~400 tables GLPI : sélection des 13 tables retenues	3
2.3	Constat : volumétrie des tables et jointures (analyse GLPI)	4
2.4	Étapes de la démarche (trace dans le dépôt)	5
2.5	Constats finaux	6
2.6	Schéma relationnel (modèle équipe)	6
3	Organisation du projet et correspondance code	7
3.1	Volet 1 — Architecte BDD	7
3.2	Volet 2 — Développeur PL/SQL	7
3.3	Volet 3 — Distribution & performance	8
4	Infrastructure Oracle	8
4.1	Tablespaces (<code>infrastructure.sql</code>)	8
4.2	Rôles et comptes (<code>owner.sql</code> , <code>acces.sql</code>)	8
5	PL/SQL	9
5.1	Gestion des exceptions	9
5.2	Séquences par campus (<code>sequences.sql</code>)	9
5.3	Triggers : rôle métier et technique (<code>triggers.sql</code>)	9
5.4	Fonctions métier (<code>fonctions/</code>)	9
5.5	Procédures métier (14) — réduction de charge utilisateur	9
5.6	Vues et vues matérialisées	11
5.7	Jeu de test	11
6	BDDR	11
6.1	Database links et vues globales (<code>bddr_cergy.sql</code> , <code>bddr_pau.sql</code>)	11
6.2	Écritures et lectures distantes	11
7	Tests de performance et plans d'exécution	11
7.1	Comparatif 11 procédures : BDD vierge vs BDD optimisée	12
7.2	Plans d'exécution détaillés (<code>explain_plan.sql</code>)	12
7.3	Déroulé du scénario (<code>scenario.sql</code>)	14
7.4	Indicateurs attendus après installation (<code>install.sql</code>)	15

8	Arborescence des livrables SQL	15
9	Exécution (ordre vérifié dans les scripts)	15

1 Contexte et périmètre

Ce mini-projet repense une partie du schéma GLPI pour le parc informatique de CY Tech (campus Cergy et Pau). Nous sommes partis du dépôt officiel GLPI (<https://github.com/glp-i-project/glpi>) et de la documentation des tables sur https://dbdocs.io/glpi/GLPI-Main?table=glpi_tickets&schema=public&view=table_structure, en limitant le périmètre au matériel, aux utilisateurs et au réseau. Le modèle Oracle livré compte **13 tables** (12 dans `schema.sql` + `glpi_history`). L'analyse GLPI et la sélection des 13 tables sont détaillées en section 2. Le déploiement passe par `install.sql` ([1/8] à [8/8], dont `donnees_initiales.sql`); `scenario.sql` valide ensuite la démo métier ; les EXPLAIN PLAN sont dans `explain_plan.sql` (section 7.2).

2 Reverse engineering

2.1 Sources et périmètre analysé

- Dépôt GLPI : <https://github.com/glp-i-project/glpi>.
- Documentation structurée des tables : https://dbdocs.io/glpi/GLPI-Main?table=glpi_tickets&schema=public&view=table_structure (navigation vers `glpi_users`, `glpi_computers`, etc.).
- Périmètre sujet : matériels, utilisateurs, réseaux — hors finance, plugins et périmètre GLPI complet (~400 tables analysées, 13 retenues : section 2.2).
- Modèle Oracle livré : **12 tables** dans `schema.sql` + `glpi_history` (`triggers.sql`).

2.2 Parcours des ~400 tables GLPI : sélection des 13 tables retenues

Le schéma documenté sur <https://dbdocs.io/glpi/GLPI-Main> compte environ **400 tables** `glpi_*`. Notre travail de reverse engineering a consisté à **parcourir l'ensemble de ce schéma** (dbdocs, dépôt GitHub, écrans métier du sujet) pour ne retenir que le noyau utile à CY Tech : parc matériel sur deux campus, utilisateurs/profils, réseau IP et tickets d'incident.

Méthode de filtrage.

1. **Inventaire** : parcours table par table sur dbdocs ; repérage des dépendances (`entities_id`, `itemtype/items_id`, tables de liaison).
2. **Conserver** si la table est indispensable à au moins un cas d'usage du sujet : localiser un équipement, affecter un technicien, ouvrir un ticket, gérer un profil, adresser une IP.
3. **Écarter** les domaines hors périmètre CY Tech, par exemple :
 - Finances / contrats / licences : `glpi_contracts`, `glpi_licenses`, `glpi_softwares`, `glpi_softwareversions`...
 - Helpdesk avancé : `glpi_ticketusers`, `glpi_tickettasks`, `glpi_itilsolutions`, `validations`, `SLA`...
 - Autres familles de matériel non déployées : `glpi_monitors`, `glpi_phones`, `glpi_network equipments`...
 - Plugins, réservations, base de connaissances, projets : centaines de tables sans lien avec notre périmètre.
4. **Simplifier** chaque table conservée (voir tableau ci-dessous) puis valider par itérations (`ourBddVierge.sql` → `schema.sql`, déployé via `install.sql`).

Bilan : 13 tables (12 dans `schema.sql` + `glpi_history` dans `triggers.sql`). Pour chaque table retenue, nous documentons le **rôle**, le **changement** par rapport à GLPI et la **raison** du choix CY Tech.

Table	Rôle	Changement vs GLPI	Raison
<code>glpi_entities</code>	Campus	Arbre hiérarchique de sous-entités → 2 lignes fixes (Cergy, Pau).	Modèle multi-sites CY Tech sans sous-organisations ; simplifie partitions et droits.
<code>glpi_locations</code>	Salles	Arbre <code>locations_id</code> (bâtiment / étage / salle) → liste plate par campus.	Pas de hiérarchie de bâtiments dans le sujet ; requêtes et affectations plus simples.
<code>glpi_users</code>	Personnes métier	Compte applicatif GLPI → ligne métier séparée des comptes Oracle.	600 lambdas sans login base ; seuls admin/technicien ont un compte DB.
<code>glpi_profiles</code>	Profils	Catalogue GLPI complet → 3 profils (admin, technicien, lecture).	Aligné sur les rôles Oracle <code>R_GLPI_*</code> .
<code>glpi_profiles_users</code>	Lien user-profil-site	Droits récursifs possibles → lien sans is_recursive , borné au campus.	Un technicien n'hérite pas des droits de l'autre site.
<code>glpi_profilerights</code>	Matrice de droits	Droits GLPI fins → table documentaire (vues / procédures du projet).	Trace les GRANT réels définis dans <code>acces.sql</code> .
<code>glpi_networks/glpi_ipaddresses</code>	Réseau	Multiplés réseaux, ports, câblage → 1 réseau / campus , IP liée au site.	Plan d'adressage fixe 10.1.0.0/24 et 10.2.0.0/24 ; génération par séquences.
<code>glpi equipments</code>	Équipement (mère)	Table générique GLPI conservée ; <code>itemtype</code> limité à Computer / Printer.	Point d'entrée unique pour site, salle, IP ; base du partitionnement.
<code>glpi_computers/glpi_printers</code>	Détail par type	Tables filles GLPI → PARTITION BY REFERENCE sur la mère.	Co-localisation physique Cergy/Pau ; même campus que <code>glpi equipments</code> .
<code>glpi_tickets</code>	Incidents	Liens multiples acteurs → ticket lié à un équipement (<code>equipment_id</code>) et un campus.	Cas d'usage lambda + technicien ; moins de jointures que le helpdesk complet.
<code>glpi_history</code>	Audit	Table ajoutée (absente du noyau minimal initial).	Traçabilité des modifications via <code>tr_hist_*</code> et <code>pkg_audit</code> .

Pourquoi `glpi equipments` → `computers/printers` ? GLPI sépare déjà les types de matériel. Nous conservons une **table mère** (`glpi equipments`) pour le site, la salle et l'IP, et des **tables filles** pour les attributs spécifiques (série, technicien, état). Cela permet le **PARTITION BY REFERENCE** : chaque PC ou imprimante reste physiquement sur le même campus que sa ligne mère.

Tables GLPI écartées (exemples). Sur les ~387 tables non retenues : tout le module logiciels/-contrats, le helpdesk étendu (`glpi_ticketusers...`), les types de matériel hors PC/imprimante, les plugins et tables de configuration avancée. Elles restent dans GLPI officiel ; notre modèle Oracle se limite au périmètre du mini-projet.

2.3 Constat : volumétrie des tables et jointures (analyse GLPI)

Dans GLPI, le module inventaire/helpdesk s'appuie sur un **grand nombre de tables** (schéma complet : centaines de tables `glpi_*`, plugins, relations polymorphes `itemtype/items_id`). Même en limitant le périmètre au sujet, une requête métier typique enchaîne de nombreuses jointures :

- **Équipements + type** : accès au matériel via une table générique et/ou des tables par type (`glpi_computers`, `glpi_printers`, `glpi_monitors...`).
- **Multi-sites** : filtre ou jointure sur `glpi_entities` (souvent hiérarchique).

- **Réseau** : `glpi_ipaddresses` → `glpi_networks` (et parfois ports, domaines, câblage selon les écrans GLPI).
- **Tickets** : `glpi_tickets` lié à l'équipement, au demandeur, au lieu, aux techniciens (`glpi_ticketusers`, groupes, tâches selon les écrans).
- **Utilisateurs / droits** : `glpi_users`, `glpi_profiles`, `glpi_profiles_users`, `glpi_profilerights`.

Exemple observé dans notre code (modèle déjà simplifié). Sans compter les tables GLPI exclues du périmètre, les vues du projet illustrent déjà des requêtes lourdes :

Objet	Tables jointes	Détail
<code>v_admin_tickets_en_retard</code>	7	<code>tickets</code> , <code>entities</code> , <code>equipments</code> , <code>locations</code> , <code>users</code> , <code>computers</code> , <code>printers</code>
<code>v_tech_tickets_actifs</code>	8+	CTE : <code>users</code> , <code>computers/printers</code> , <code>equipments</code> ; puis <code>entities</code> , <code>locations</code> , <code>ipaddresses</code> , <code>tickets</code> , <code>users</code>
<code>rapport_utilisateurs_site</code> (<code>cursors.sql</code>)	7	<code>users</code> , <code>profiles_users</code> , <code>profiles</code> , <code>computers</code> , <code>printers</code> , <code>equipments</code> , <code>locations</code>
<code>mv_charge_techniciens</code>	5	<code>users</code> , <code>computers</code> , <code>printers</code> , <code>equipments</code> , <code>entities</code>

Conséquences pour la refonte CY Tech.

- Le modèle Oracle conserve l'héritage `equipments` + tables filles : les jointures `computers/printers` restent nécessaires pour le technicien, l'état (`states_id`) et le demandeur.
- **Réponse projet** : encapsuler les jointures dans des **vues / vues matérialisées** (`vues_logiques/`, `vues_materialisees/`) et interdire l'accès DML direct aux tables (`acces.sql`).
- Comparaison perf documentée : requête de construction `MV_PARC` vs lecture `mv_read_parce_par_site` dans `explain_plan.sql` (section 7.2).

2.4 Étapes de la démarche (trace dans le dépôt)

Le reverse engineering n'est pas un document externe : il est **incrémental** et visible dans l'évolution des scripts SQL.

Étape 1 — Inventaire et traduction DDL minimale. Fichier `ourBddVierge.sql` (en-tête : *VERSION ULTRA-VIERGE / SANS INDEX*) :

- Reprise des noms de tables GLPI (`glpi_*`).
- Création de **12 tables** + clés étrangères + CHECK (`states_id`, `status`, `entities_id` IN (1,2)); pas de `glpi_history` (DROP défensif seulement).
- Pas encore de tablespaces, partitions, index, triggers, procédures.
- INSERT fixes CERGY/PAU dans `glpi_entities`.
- Objectif : valider le modèle relationnel pur avant l'optimisation Oracle.

Étape 2 — Simplifications et modèle final (`schema.sql`).

- **Lieux plats** : plus de `locations_id` parent sur `glpi_locations`; `lvl=0`; salles par campus (LOC-1 à LOC-16 dans `donnees_initiales.sql`).
- **Entités plates** : deux sites fixes (CERGY, PAU) sans sous-entités.
- **Profils** : pas de colonne `is_recursive` sur `glpi_profiles_users`; droits limités au `entities_id` du profil.
- **Matériel** : `glpi_equipments` (mère) + `glpi_computers` / `glpi_printers` (filles), `itemtype` limité à `Computer` ou `Printer`.
- **Comptes** : ligne `glpi_users` ≠ compte Oracle; lambdas sans `glpi_profiles_users` ni compte DB.
- **Session** : `CLIENT_IDENTIFIER` au format `PSEUDO|SITE` (ex. `ADUPONT|CERGY`).
- **Réseau** : `glpi_ipaddresses` reliée au site via `networks_id` (plus de `entities_id` sur la table IP).
- **Unicité lieux** : index `uk_loc_ent_name` sur (`entities_id`, `name`).

Étape 3 — Comptes Oracle et métier (owner.sql, acces.sql).

- Comptes techniciens/admins **créés dynamiquement** (ajouter_technicien, ajouter_admin).
- Comptes partagés : GLPI_READ, GLPI_HELP.
- Utilisateur métier (glpi_users) $\neq$ compte Oracle (voir schema.sql).

Étape 4 — Cartographie des droits GLPI → Oracle. `donnees_initiales.sql`, parties « 1. *PROFILS GLPI* » et « 2. *DROITS PAR PROFIL* » : les lignes `glpi_profilerights` documentent les vues/procédures ; les droits réels sont les **GRANT** dans `acces.sql`.

Étape 5 — Jeu de données et performance.

- Données de test : `donnees_initiales.sql` (partie 5).
- Tests de performance : comparatif `ourBddVierge.sql` / `explain_plan.sql` (section 7) et démo `scenario.sql`.

Étape 6 — Déploiement. `install.sql` orchestre `infrastructure.sql`, `schema.sql`, PL/SQL, vues, `acces.sql`, `donnees_initiales.sql`.

2.5 Constats finaux

- Modèle cible : **12 tables** dans `schema.sql` + `glpi_history` dans `triggers.sql` (schéma GLPI complet bien plus large).
- Livrable d'installation : `install.sql` uniquement (`README.md`).
- Réseau simplifié : 1 réseau / site (10.1.0.0/24, 10.2.0.0/24) ; IPs générées par `seq_ip_host_*` dans `ajouter_equipement`.
- Contexte applicatif simulé : `DBMS_SESSION.SET_IDENTIFIER('PSEUDO|CERGY')` (`scenario.sql`, `donnees_initiales.sql`).

2.6 Schéma relationnel (modèle équipe)

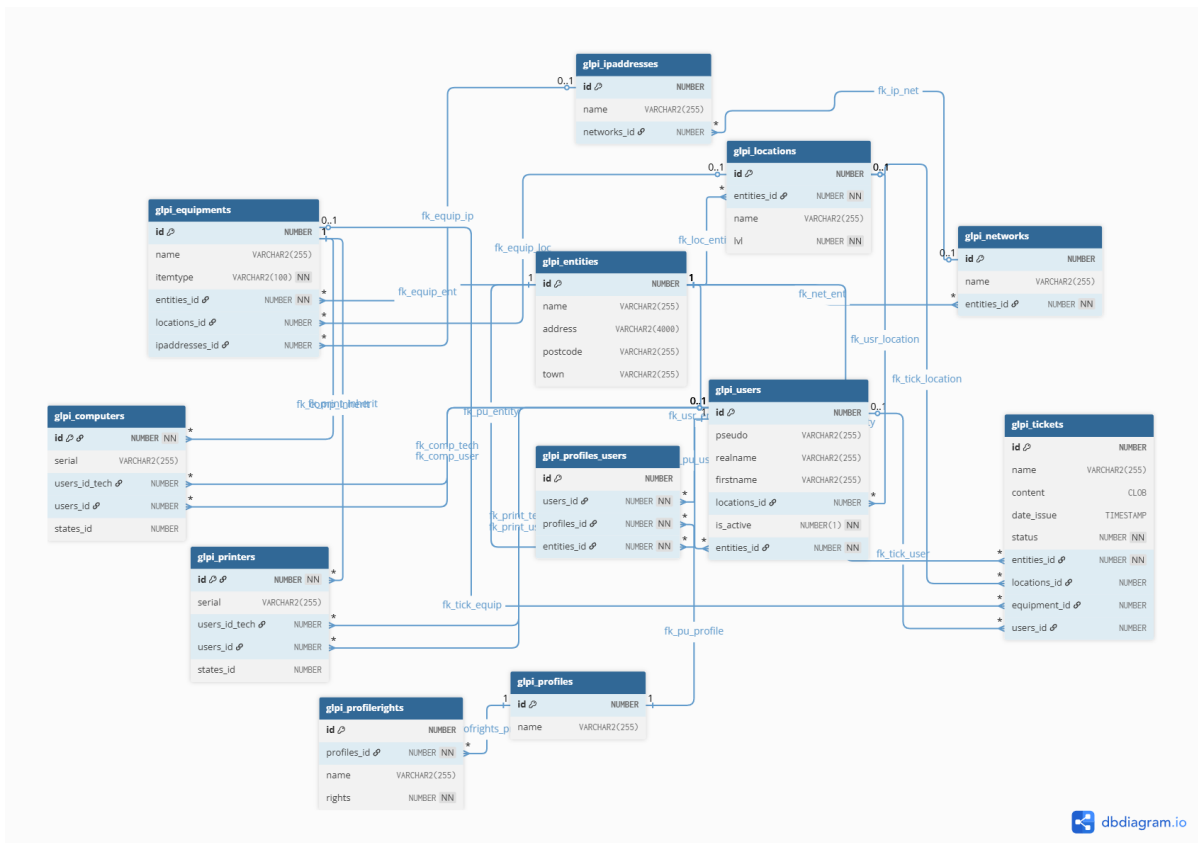
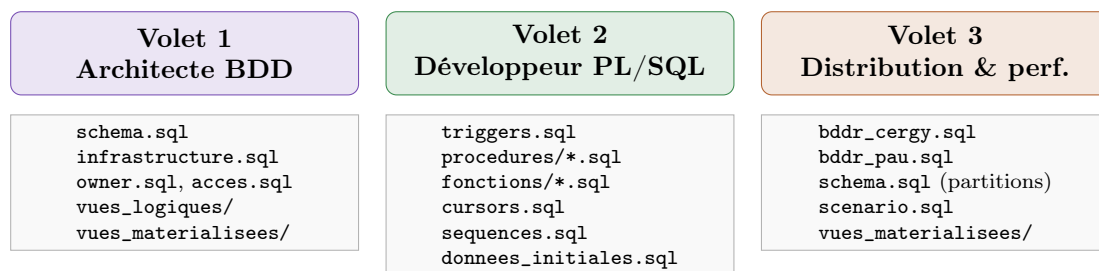


FIGURE 1 – Schéma de la base CY Tech (travail de modélisation équipe).

3 Organisation du projet et correspondance code

Le mini-projet repose sur trois volets techniques (architecture Oracle, PL/SQL métier, distribution et performances), alignés sur l’infographie de travail de l’équipe. Le dépôt GitHub découpe le livrable en scripts modulaires ; la figure et les tableaux ci-dessous indiquent quel fichier couvre chaque livrable du sujet. Le reverse engineering (section 2) constitue la première étape du projet ; les sections suivantes détaillent l’implémentation Oracle.



3.1 Volet 1 — Architecte BDD

Livrable	Fichiers / contenu
Reverse engineering	Parcours ~400 tables dbdocs → 13 retenues (section 2) ; schema.sql.
Schéma relationnel	13 tables, PK/FK/CHECK ; diagramme équipe (figure 1).
Users & rôles	infrastructure.sql, owner.sql, acces.sql.
Tablespaces	TS_GLPI_REF, TS_GLPI_CERGY, TS_GLPI_PAU, TS_GLPI_INDX.
Co-localisation	PARTITION BY REFERENCE : tables filles sur le même site que glpi_equipements (voir détail ci-dessous).
Index	Bloc « 3. INDEX SECONDAIRES » de schema.sql ; index LOCAL.
Vues	vues_logiques/ (5), vues_materialisees/ (2).

3.2 Volet 2 — Développeur PL/SQL

Livrable	Fichiers (détail section 5)
Triggers intégrité / historisation	triggers.sql — cohérence campus + glpi_history.
Procédures / fonctions	procedures/ (14), fonctions/ (3) ; NOT EXISTS dans repartition_charge_nouveau_tech.
Curseurs	cursors.sql (rapports par site, hors install.sql).
Jeu de test	donnees_initiales.sql partie 5 (chargé par install.sql) ; scenario.sql = démo seule.
Séquences	sequences.sql — noms d’équipements, tickets, IP par campus.
Gestion exceptions	RAISE_APPLICATION_ERROR dans triggers ; blocs EXCEPTION dans procédures (ex. creer_ticket.sql, supprimer_admin.sql) ; COMMIT/ROLLBACK en cas d’erreur.
PL/SQL	

3.3 Volet 3 — Distribution & performance

Livrable infographie	Implémentation (fichier)
BDDR : DB Links	bddr_cergy.sql : CREATE PUBLIC DATABASE LINK dblink_vers_pau.
bddr_pau.sql : lien symétrique vers Cergy.	
Comptes GLPI_DBLINK_* dans owner.sql.	
Vues réparties Cergy/Pau	v_global_equipements, v_global_users, v_global_tickets (UNION ALL local + @dblink). Grants dans acces.sql.
Partitionnement par site	schema.sql : PARTITION BY LIST (entities_id) sur glpi_locations, glpi_equipments, glpi_tickets.
Tests de performance	explain_plan.sql (vues, MV, fonctions, 14 proc.); comparatif ourBddVierge.sql vs optimisé (section 7.1).
Comparaison avant/après	11 procédures mesurées (tableau Excel); récapitulatif SUM(cost) en fin de explain_plan.sql.

4 Infrastructure Oracle

4.1 Tablespaces (infrastructure.sql)

Tablespace	Fichier datafile	Rôle
TS_GLPI_REF	ts_glpi_ref01.dbf	Référentiels, historique
TS_GLPI_CERGY	ts_glpi_cergy01.dbf	Partitions campus Cergy
TS_GLPI_PAU	ts_glpi_pau01.dbf	Partitions campus Pau
TS_GLPI_INDX	ts_glpi_indx01.dbf	Tous les index (séparation IO)

Répartition des 13 tables (partitions). Référentiels (glpi_entities, glpi_users, profils, réseau, glpi_history) : TS_GLPI_REF, sans partition. Données métier par campus (glpi_locations, glpi_equipments, glpi_tickets) : TS_GLPI_CERGY / TS_GLPI_PAU avec PARTITIONBYLIST(entities_id). Tables filles (glpi_computers, glpi_printers) : PARTITIONBYREFERENCE sur la mère (schema.sql).

4.2 Rôles et comptes (owner.sql, acces.sql)

Rôle / compte	Usage métier	Droits techniques
R_GLPI_READ	Auditeur : consultation multi-sites	Vues v_read_*, mv_read_parc_par_site.
GLPI_READ	Compte Oracle partagé auditeur	Reçoit le rôle R_GLPI_READ (ce n'est pas un second rôle).
R_GLPI_TECH	Technicien : ses équipements / tickets	v_tech_tickets_actifs + 4 procédures.
R_GLPI_ADMIN	Admin campus : users + équipements	12 procédures + vues admin.
R_GLPI_TICKET_HELP	Helpdesk : déclarer un incident	EXECUTE sur creer_ticket seul.
GLPI_HELP	Compte Oracle helpdesk	Reçoit R_GLPI_TICKET_HELP (rôle + compte, pas un doublon).
GLPI_OWNER	Déploiement	Propriétaire schéma (hors usage métier).
GLPI_DBLINK_*	Service BDDR	Lecture distante pour vues v_global_*.

Principe du moindre privilège (acces.sql). Aucun GRANT INSERT/UPDATE/DELETE sur les tables vers les rôles applicatifs : accès uniquement par vues et procédures (AUTHID DEFINER).

Exemple — compte technique helpdesk (GLPI_HELP / R_GLPI_TICKET_HELP). Si l'application web est compromise et que l'attaquant vole la session du compte technique (équivalent R_GLPI_WEB_APP / GLPI_HELP), il peut au plus créer des tickets en boucle via `creer_ticket`. Il ne peut pas modifier les tables, supprimer des données (DELETE) ni s'octroyer des droits administrateur : le compte n'a que EXECUTE sur cette procédure. C'est le **principe du moindre privilège**, argument central pour la maintenance.

5 PL/SQL

5.1 Gestion des exceptions

RAISE_APPLICATION_ERROR dans les triggers ; blocs EXCEPTION dans les procédures (ex. `creer_ticket.sql`, `supprimer_admin.sql`). Codes métier -20001 à -20053 (pseudo déjà pris, Last Man Standing sur `supprimer_admin`, etc.).

Les procédures regroupent les écritures dans un bloc transactionnel : en cas d'erreur, ROLLBACK annule la transaction ; sinon COMMIT valide (ex. `ajouter_admin`, `creer_ticket`, `supprimer_technicien`). Les blocs EXCEPTION WHEN OTHERS THEN ROLLBACK ; RAISE ; évitent toute écriture partielle en base.

5.2 Séquences par campus (sequences.sql)

Les séquences `seq_equip_cergy/seq_equip_pau`, `seq_ticket_*` et `seq_ip_host_*` produisent des identifiants **automatiques et uniques par site** (noms d'équipement EQ-10001, tickets, hôtes IP 10.1.0.x / 10.2.0.x). L'application n'a pas à calculer les numéros : la procédure `ajouter_equipement` consomme la bonne séquence selon le campus.

5.3 Triggers : rôle métier et technique (triggers.sql)

Trigger	Pourquoi (métier)	Technique
<code>tr_ticket_equip_entity</code>	Un ticket reste sur le campus de son équipement	BEFOREINSERT/UPDATE
<code>tr_equip_loc_entity</code>	Pas d'équipement dans une salle d'un autre site	Cohérence campus
<code>tr_*_no_chg_entity</code>	Interdit de « déplacer » une ligne vers l'autre partition	Intégrité multi-sites
<code>tr_hist_*</code>	Traçabilité des modifications	<code>pkg_audit</code> → <code>glpi_history</code>

5.4 Fonctions métier (fonctions/)

`repartition_charge_nouveau_tech` — **métier et technique**. **Métier** : lors de l'embauche d'un technicien, répartir équitablement les équipements entre techniciens du campus, sans retirer un matériel avec ticket bloquant.

Technique : curseur + NOT EXISTS (sous-requête corrélée sur `glpi_tickets` avec `status` IN (2,3)) ; retourne une liste d'IDs (`t_ids`) consommée par `ajouter_technicien`.

`get_equip_technicien` et `redistribuer_equip` complètent les affectations d'équipements.

5.5 Procédures métier (14) — réduction de charge utilisateur

Les procédures encapsulent les règles : l'application n'écrit plus directement dans les tables. Contrôle des données (campus, pseudo unique, cohérence équipement–ticket). Requêtes dynamiques (NOT EXISTS, sous-requêtes corrélées) dans les fonctions et certaines procédures.

Procédure	Métier / technique	Note
ajouter_equipement	Métier : enregistrer un nouvel équipement sur un campus. Technique : INSERT mère + fille ; IP via séquences.	seq_equip_*, seq_ip_host_* ; noms EQ-10001, 10.1.0.x...
affecter_localisation_equipement	Métier : rattacher une salle à un équipement. Technique : contrôle entities_id équipement / salle.	Évite un matériel Cergy dans une salle Pau.
affecter_technicien_equipement	Métier : désigner le technicien responsable. Technique : UPDATEusers_id_tech sur computers/printers.	Base des vues v_tech_tickets_actifs.
changer_statut_equipement	Métier : marquer un équipement en panne / réparé. Technique : UPDATEstates_id avec contrôles campus.	Prérequis avant creer_ticket.
creer_ticket	Métier : ouvrir un incident sur un équipement identifié. Technique : jointure equipments + fille ; COMMIT/ROLLBACK.	Seule action du compte GLPI_HELP.
modifier_statut_ticket	Métier : faire avancer le traitement (en cours, résolu...). Technique : UPDATEstatus ; vérifie technicien / campus.	Workflow helpdesk sans SQL manuel.
ajouter_admin	Métier : nommer un administrateur de site. Technique : CREATEUSER + glpi_profiles_users.	Compte Oracle dédié au campus.
ajouter_technicien	Métier : embaucher un technicien et équilibrer la charge. Technique : appelle repartition_charge_nouveau_tech.	Réaffectation automatique d'équipements.
ajouter_utilisateur_lambda	Métier : créer un étudiant / utilisateur sans accès DB. Technique : INSERT sur glpi_users seul.	Pas de ligne glpi_profiles_users.
ajouter_lambda_autre_site	Métier : activer un lambda sur l'autre campus. Technique : copie locale ; pseudo dérivé.	Pas de compte Oracle.
ajouter_tech_ou_admin_autre_site	Métier : permettre l'intervention sur les deux sites. Technique : copie + CREATEUSER + profil.	Complète la BDDR en lecture.
supprimer_admin	Métier : retirer un admin (désactivation). Technique : soft delete ; Last Man Standing (-20053).	Garde au moins un admin actif par site.
supprimer_technicien	Métier : retirer un technicien. Technique : réaffectation des équipements ; ROLLBACK si échec.	Évite les équipements orphelins.
supprimer_utilisateur_lambda	Métier : désactiver un lambda. Technique : is_deleted=1 ; pas de drop Oracle.	Désactivation logique uniquement.

Copies cross-campus (ajouter_lambda_autre_site, ajouter_tech_ou_admin_autre_site). **Métier** : un étudiant ou un technicien peut être actif sur les deux campus sans ressaisie manuelle.

Technique : recopie sur l'instance courante avec pseudo dérivé et contrôle d'unicité ; pour un tech/admin, création du compte Oracle et du lien glpi_profiles_users. La BDDR (v_global_*) complète par la lecture inter-instances ; les INSERT @dblink restent une extension possible non codée dans les procédures livrées.

5.6 Vues et vues matérialisées

- `v_tech_tickets_actifs` : le technicien ne voit que **ses** équipements et leurs tickets ouverts (`CLIENT_IDENTIFIER`) — évite des jointures manuelles côté client.
- Vues admin : retard, inactifs, charge — filtrage par campus pour la supervision.
- `mv_read_parc_par_site` / `mv_charge_techniciens` : agrégats `REFRESH COMPLETE`; coûts comparés dans `explain_plan.sql` (sections 2 et récapitulatif).
- `v_read_tickets_non_resolus` : vue auditeur multi-sites sans filtre session.

5.7 Jeu de test

Fichier des données de test : `donnees_initiales.sql` (partie « 5. *DONNEES DE TEST* »), exécuté automatiquement à la fin de `install.sql`.

Ne pas confondre avec `scenario.sql` : ce dernier enchaîne une démonstration métier (ACTE 1–8), pas la génération initiale du parc.

Élément	Volume
Équipements par campus (PC + imprimantes)	75 (50+25)
Tickets	20
Utilisateurs <code>glpi_users</code>	604
Adresses IP	150
Salles	16

Génération via `ajouter_equipement` en boucle (5.1–5.2); affectation des techniciens aux équipements (5.5).

6 BDDR

6.1 Database links et vues globales (`bddr_cergy.sql`, `bddr_pau.sql`)

Sur l'instance Cergy :

```
CREATE PUBLIC DATABASE LINK dblink_vers_pau ...
CREATE VIEW v_global_equipements AS
  SELECT ... FROM glpi_equipements WHERE entities_id = 1
  UNION ALL
  SELECT ... FROM glpi_equipements@dblink_vers_pau WHERE entities_id = 2;
```

Même principe pour `v_global_users` et `v_global_tickets` : **UNION ALL** entre données locales et lecture distante via `@dblink`.

6.2 Écritures et lectures distantes

- **Lecture** : vues `v_global_*` avec `UNION ALL` entre données locales et `SELECT ...@dblink_vers_pau` (ou Cergy).
- **Écriture distante (extension possible)** : pattern `INSERT INTO ...@dblink_vers_pau` pour répliquer une ligne sur l'autre instance; non implémenté dans les procédures livrées (copies cross-campus faites localement via `ajouter*_autre_site`).

Activation : `@bddr_cergy.sql` ou `@bddr_pau.sql` en fin de `install.sql`.

7 Tests de performance et plans d'exécution

Les mesures reposent sur `EXPLAIN PLAN` et `DBMS_XPLAN.DISPLAY` (coût CPU, cardinalité). Dépôt : <https://github.com/timfrncs/ing2-advanced-db-glpi-redesign>.

7.1 Comparatif 11 procédures : BDD vierge vs BDD optimisée

Protocole (aligné sur la dernière version du dépôt).

- **Sans optimisation** : section EXPLAIN PLAN de `ourBddVierge.sql` — schéma vierge sans index secondaires (full table scans).
<https://github.com/timfrncs/ing2-advanced-db-glpi-redesign/blob/main/ourBddVierge.sql> (à partir de la section *Plans d'exécution*).
- **Avec optimisation** : mêmes requêtes dans `explain_plan.sql`, exécuté sur la BDD déployée par `install.sql` (schema.sql : index LOCAL, partitions, vues, procédures).
https://github.com/timfrncs/ing2-advanced-db-glpi-redesign/blob/main/explain_plan.sql.
- **Synthèse visuelle** : tableau de bord Excel (`screen/tab_final.png`, `diff_cout_2.png`, `imp_opti.png`, `diff_cout.png`, `cout-req.png`).
- **Périmètre Excel** : 11 procédures (P01–P14 sauf `affecter_localisation_equipement`, `affecter_technicien_equipement`, `changer_statut_equipement`) ; les 14 procédures sont détaillées dans `explain_plan.sql`.

ID	Type de requête testé	Cout sans opti	Cout avec opti	Ecart
1	ajouter_admin	26	9	17
2	ajouter_equipement	8	9	-1
3	ajouter_lambda_autre_site	47	10	37
4	ajouter_tech_ou_admin_autre_site	49	15	34
5	ajouter_technicien	561	9	552
6	ajouter_utilisateur_lambda	23	5	18
7	creer_ticket	128	9	119
8	modifier_statut_ticket	54	16	38
9	supprimer_admin	54	16	38
10	supprimer_technicien	918	12	906
11	supprimer_utilisateur_lambda	42	11	31

FIGURE 2 – Tableau synthèse : coût CPU des 11 procédures testées (sans opti / avec opti / écart).
Source : `screen/tab_final.png`.

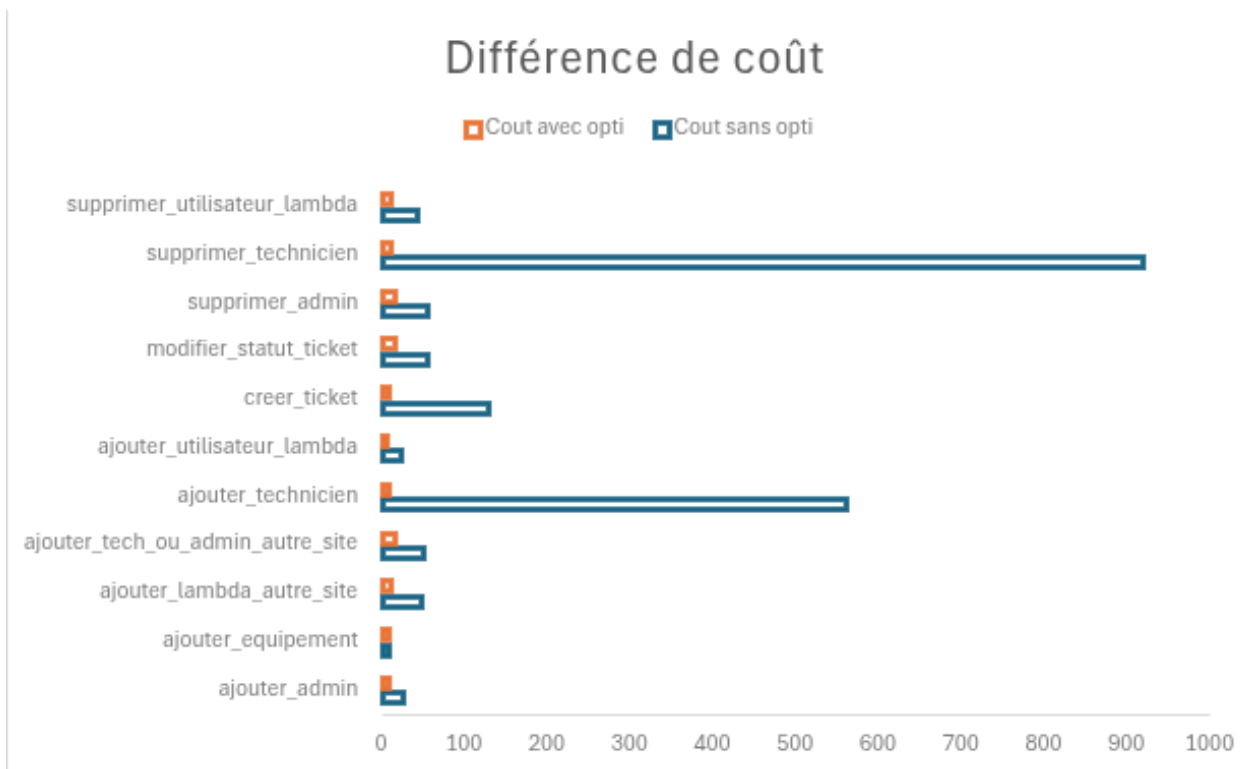
Lecture : les gains les plus marqués concernent `supprimer_technicien` (écart 906) et `ajouter_technicien` (écart 552). Seul `ajouter_equipement` augmente légèrement (+1) du fait des contrôles métier ajoutés dans la procédure optimisée.

7.2 Plans d'exécution détaillés (`explain_plan.sql`)

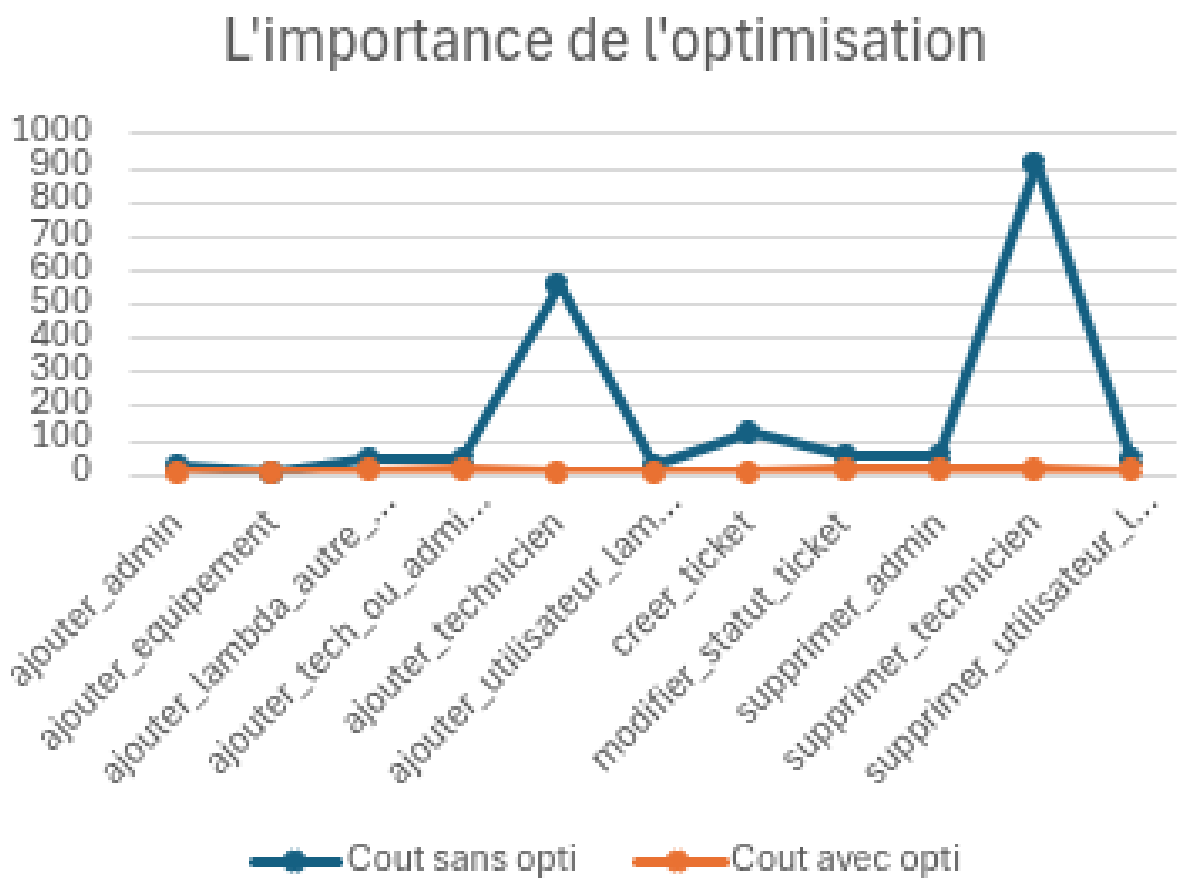
Fichier dédié : https://github.com/timfrncs/ing2-advanced-db-glpi-redesign/blob/main/explain_plan.sql.

Prérequis : `install.sql` + `donnees_initiales.sql` (GLPI_OWNER). **Non inclus** dans `install.sql` : à lancer manuellement après installation.

Section	Objets couverts
1 — Vues logiques	<code>v_tech_tickets_actifs</code> , <code>v_admin_equipements_inactifs</code> , <code>v_admin_charge_techniciens</code>
2 — Vues matérialisées	<code>mv_charge_techniciens</code> , <code>mv_read_parc_par_site</code>
3 — Fonctions	<code>get equip_technicien</code> , <code>redistribuer equip</code> , <code>repartition_charge_nouveau_tech</code>
4 — Procédures (14)	<code>ajouter_admin</code> (P01) à <code>supprimer_technicien</code> (P14) : chaque requête SQL interne a un <code>STATEMENT_ID</code> (P08_SEL_EQ, etc.)
Récapitulatif	<code>SELECT ... SUM(cost) ... GROUP BY</code> sur <code>plan_table</code> : coût total par objet, tri décroissant

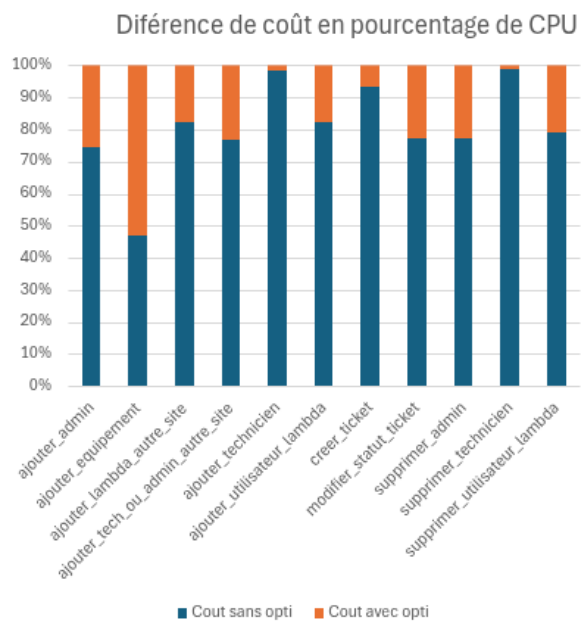


(a) Différence de coût (barres horizontales).

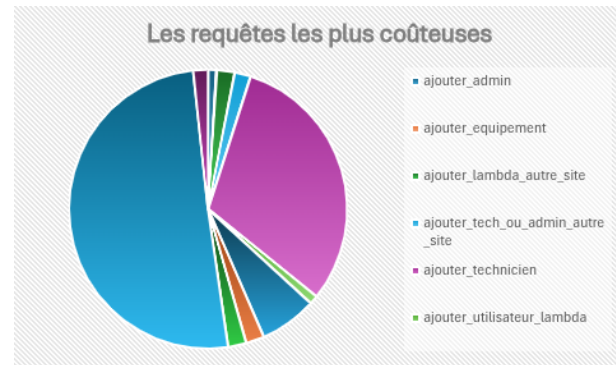


(b) L'importance de l'optimisation (courbes).

FIGURE 3 – Comparaison des coûts : BDD vierge (`ourBddVierge.sql`) vs BDD optimisée (`explain_plan.sql`).



(a) Différence de coût en pourcentage de CPU.



(b) Les requêtes les plus coûteuses.

FIGURE 4 – Répartition relative des coûts (screen/diff_cout.png, screen/cout-req.png).

7.3 Déroulé du scénario (scenario.sql)

Fichier : <https://github.com/timfrncs/ing2-advanced-db-glpi-re-design/blob/main/scenario.sql>.

Prérequis : install.sql + donnees_initiales.sql ; connexion GLPI_OWNER.

Contexte session : DBMS_SESSION.SET_IDENTIFIER('PSEUDO|SITE') (ex. ADUPONT|CERGY, FMARTIN|CERGY, CBERNARD|PAU, AUDIT|READ).

Plans d'exécution : absents de ce fichier — voir explain_plan.sql (section 7.2) ; le script se termine par un PROMPT de renvoi.

Personnages. ADUPONT (admin Cergy, existant) pilote les actes Cergy ; créations de démo : EPERON, JLEFEVRE, FMARTIN, TDEMO ; BMARTIN (tech existant) ; copies cross-site depuis Pau : UP005_CERGY, CBERNARD_CERGY ; sur Pau : UP005, CBERNARD (originaux intacts).

État initial. Comptage équipements par site/itemtype, puis tickets par statut (Nouveau à Résolu).

Bloc	Déroulé (aligné sur scenario.sql)
ACTE 1	ADUPONT CERGY : [1] ajouter_admin ×2 (EPERON, JLEFEVRE) ; [2] ajouter_technicien (FMARTIN) ; refresh mv_charge_techiciens [2b] + v_admin_charge_techiciens ; [3] ajouter_utilisateur_lambda (TDEMO) ; [4-6] trois ajouter_equipement (SER-DEMO-PC-001/002, SER-DEMO-PR-001), affecter_localisation_equipement ×3, affecter_technicien_equipement ×3 vers FMARTIN ; [7] changer_statut_equipement ×2 (PC en service, imprimante en stock).
ACTE 2	TDEMO : [8] creer_ticket sur SER-DEMO-PC-001 (écran noir).
ACTE 3	FMARTIN CERGY : v_tech_tickets_actifs ; [9] modifier_statut_ticket ×2 (Nouveau→En cours→Résolu) ; nouvelle lecture v_tech_tickets_actifs.
ACTE 4	ADUPONT CERGY : v_admin_tickets_en_retard, v_admin_equipements_inactifs ; lecture mv_charge_techiciens (état pré-refresh) ; DBMS_MVIEW.REFRESH('mv_charge_techiciens','C') ; relecture MV ; v_admin_charge_techiciens (filtre site via CLIENT_IDENTIFIER).
ACTE 5	ADUPONT CERGY : [10] ajouter_lambda_autre_site('UP005') → UP005_CERGY ; [11] ajouter_tech_ou_admin_autre_site('CBERNARD',...) → CBERNARD_CERGY.

Bloc	Déroulé (aligné sur scenario.sql)
ACTE 6	Pau : [8] creer_ticket par UP005 sur EQ-20001; CBERNARD PAU : [9] modifier_statut_ticket → En cours; v_tech_tickets_actifs pour CBERNARD.
ACTE 7	AUDIT READ : v_read_tickets_non_resolus; DBMS_MVIEW.REFRESH('mv_read_parc_par_site','C'); lecture mv_read_parc_par_site.
ACTE 8	ADUPONT CERGY : [12] supprimer_utilisateur_lambda (TDEMO, UP005_CERGY); [13] supprimer_technicien (FMARTIN, CBERNARD_CERGY); [14] supprimer_admin (EPERON; JLEFEVRE reste — Last Man Standing).
Bilan	Requête union : volumes équipements Cergy/Pau, tickets par statut, admins/techniciens actifs par site; PROMPT récapitulatif 14/14 procédures et vues utilisées; aucun bloc EXPLAIN PLAN — renvoi vers explain_plan.sql.

7.4 Indicateurs attendus après installation (install.sql)

Table	Attendu
glpi_entities	2
glpi_profiles	3
glpi_networks	2
glpi_users	604
glpi_profiles_users	4
glpi_profilerights	>0 (72 insérés; bilan affiche 0 = placeholder)
glpi_ipaddresses	150
glpi equipments	150
glpi_computers	100
glpi_printers	50
glpi_tickets	20
glpi_locations	16
glpi_history	>0 après usage (placeholder 0 dans bilan)

8 Arborescence des livrables SQL

```

install.sql          -- [1/8] a [8/8] + donnees_initiales + BDDR (opt.)
infrastructure.sql   -- tablespaces + roles
owner.sql            -- comptes Oracle
schema.sql           -- DDL + index + FK
sequences.sql
triggers.sql
procedures/          -- 14 procedures
fonctions/           -- 3 fonctions
vues_logiques/       -- 5 vues
vues_materialisees/  -- 2 MV
acces.sql            -- grants + synonymes
donnees_initiales.sql
explain_plan.sql     -- EXPLAIN PLAN (BDD optimisee, apres install)
ourBddVierge.sql     -- EXPLAIN PLAN (BDD vierge sans index, comparatif)
scenario.sql         -- demonstration metier ACTE 1-8
bddr_cergy.sql / bddr_pau.sql
cleanup.sql

```

9 Exécution (ordre vérifié dans les scripts)

1. **Installation** (SYSDBA) : @install.sql — enchaîne infrastructure, owner, schéma, PL/SQL, vues, droits, donnees_initiales.sql.

2. **BDDR** (optionnel, après install) : sur chaque instance, `@bDDR_cergy.sql` ou `@bDDR_pau.sql` (décommenter dans `install.sql` ou lancer à la main).
3. **Plans d'exécution** (GLPI_OWNER) : `@explain_plan.sql` — après `install.sql`, non inclus dans l'installateur.
4. **Comparatif sans index** (optionnel) : exécuter la section EXPLAIN de `ourBddVierge.sql` sur un schéma vierge.
5. **Scénario de validation** (GLPI_OWNER) : `@scenario.sql` — démo métier, non inclus dans `install.sql`.
6. **Curseurs** (optionnel) : `@cursors.sql` puis `EXEC rapport_utilisateurs_site(1);` etc.
7. **Remise à zéro** (SYSDBA) : `@cleanup.sql`